Slide 1 - Multi-Weight Molecular Spintronic Synapses for Reinforcement Learning

I am going to present my work on multi-weight molecular spintronic synapses for reinforcement learning.

The main idea is to connect two consept. On one side, we have reinforcement learning, specifically Deep Q-Learning. On the other side, we have in-memory hardware, where physical devices act like artificial synapses.

Instead of storing neural-network weights only as software numbers, we show weights represented directly by a physical conductance state. And more importantly, one physical synapse support more than one effective weight, so the same network can adapt to more than one task.

Slide 2 - The Physical Limits of Modern AI Workloads

Most modern computers are based on the von Neumann architecture. This means memory and processing are physically separated.

The processor does the calculations. The memory stores the data and the weights. The system must constantly move weights, activations, gradients, and updates back and forth. This creates what we call the memory-transfer bottleneck. The processor may be fast, but it often waits for data to arrive from memory. This movement also consumes a lot of energy.

Slide 3 - Bypassing the von Neumann Bottleneck

This slide shows the basic difference.

On the left, we have the classical model. The processor and memory are separate.

On the right, the idea changes. The physical device itself stores the weight. At the same time, it participates in the computation.

This is why in-memory computing is attractive for neural networks. Neural networks are mostly matrix operations, and matrix operations can be mapped very naturally to physical arrays of conductance devices.

Slide 4 - Moving Computation to the Data: Neuromorphic & Spintronic Alternatives

Conventional AI hardware separates storage and processing, using high-precision numbers. This ensures high accuracy but creates significant data movement.

In contrast, neuromorphic hardware stores weights directly within physical components like memristive or spintronic devices. Their conductance levels act as artificial synaptic weights.

Because these devices are non-volatile and compact—with a single nanoscale device mimicking

one synapse—the hardware can store, update, and compute data locally at the device level, greatly improving efficiency.

Slide 5 - The Core Translation: Physical Laws as Matrix Operations

This slide is the key translation between neural networks and hardware.

In a neural network, we multiply an input by a weight. We can write that as weight times input.

In hardware, the input is represented by an applied voltage. The weight is represented by conductance. The output is current.

The important formula is:

```
I = G V
```

This means current equals conductance multiplied by voltage.

Slide 6 - The Innovation: Coexisting Realities in a Single Synapse

The innovation in this work is not just using conductance as a weight. The special part is using a molecular spintronic device to allow multiple effective weights to coexist in one synapse.

A normal synapse usually gives one weight. But here, the synapse can expose different effective weights depending on the magnetic configuration.

This is important for related tasks. Instead of training completely separate neural networks, we can use one shared architecture and select task-specific weights inside the same hardware structure.

Slide 7 - The Molecular Spintronic Crosspoint Device

This hardware device features a vertical, sandwich-like structure: a Cobalt (Co) top electrode, a middle layer of AlOx and Alq3, and an LSMO bottom electrode.

It functions as a artificial synapse by using two physical effects:

- **Resistive Switching:** Electrical resistance is modified via voltage pulses.
- **Magnetoresistance:** Resistance shifts based on the magnetic alignment of the electrodes.

This combination allows the device to be programmed electrically and selected magnetically, making it effective for multi-task operations.

Slide 8 - Mechanism 1: Resistive Switching Encodes the Weight

The device changes its resistance to simulate how a brain synapse works. High resistance means a "weak" synapse, while low resistance means a "strong" one.

This relationship is simple: $G = 1 / R$. When resistance drops, conductance rises.

Learning happens by sending voltage pulses to the device:

- **SET pulse:** Increases conductance (Potentiation/strengthening).
- **RESET pulse:** Decreases conductance (Depression/weakening).

Essentially, "learning" in this hardware is just adjusting conductance levels using these electrical pulses.

Slide 9 - Mechanism 2: Magnetic Alignment Selects the Task Context

This device uses magnetic layers to add a second layer of control.

The magnetic layers can be in two states:

- **Parallel (P):** Conductance is G_P.
- **Antiparallel (AP):** Conductance is G_{AP}.

These conductances differ due to magnetoresistance. You can flip between these states using a pulse called **Spin-Orbit Torque (SOT)**, which rotates the magnetic layer without needing a big external magnet.

In short, the hardware has two separate tools:

1. **Resistive Switching:** Sets the weight value.
2. **Magnetoresistance:** Chooses which version of the weight (P or AP) is used.

This allows the synapse to store multiple "versions" of a weight, making it a powerful, multi-task tool.

Slide 10 - Anatomy of the Molecular Synapse

This design uses three components to create one "multi-weight" synapse that can switch between two tasks: one shared bias and two task-specific parts.

To get the final weight (w), the hardware performs a simple calculation:

- **For Task 0:** $w_0 = \alpha \times (G_{AP,0} + G_{P,1} - G_{bias})$
- **For Task 1:** $w_1 = \alpha \times (G_{P,0} + G_{AP,1} - G_{bias})$

How it works:

1. **The Active Task:** Uses its *antiparallel* conductance (G_{AP}), which represents the "active" weight.
2. **The Inactive Task:** Uses its *parallel* conductance (G_{P}), which acts as a background value.
3. **The Bias:** Subtracts a fixed amount (G_{bias}).

Since the hardware only produces positive conductance values, subtracting this bias is necessary to create negative weights, which are essential for standard neural networks.

Slide 11 - The Simulation Model: Capturing Non-Linear Physics

Real resistive-switching devices do not usually change in a perfectly linear way. If we apply more pulses, conductance changes, but the growth may slow down or behave nonlinearly.

To model this in PyTorch, the conductance is calculated from an internal state x using a logarithmic equation:

```
G(x) = g * log10(x)
```

Here, x represents the internal programmed state of the crosspoint. The value g is a scaling factor.

The logarithm is useful because it prevents the conductance from growing too fast. It gives a simple way to capture the non-linear nature of physical devices.

Slide 12 - Hardware-Inspired Learning: The Update Logic Tree

In standard AI, software optimizers use exact gradient values to update weights. However, physical hardware works better with simple pulses (increase or decrease) rather than high-precision numbers.

To solve this, we use the **sign of the gradient** (positive, negative, or zero):

- **Positive Gradient:** We need to decrease the weight. To do this, we increase the G_{bias}. Since we subtract the bias, this makes the final weight smaller.
- **Negative Gradient:** We need to increase the weight. To do this, we increase the conductance of the active task crosspoint, making the final weight larger.
- **Zero Gradient:** We do nothing; the hardware stays as it is.

This "sign-based" approach allows the system to learn using simple electrical pulses, making it perfectly suited for physical hardware.

Slide 13 - Full System Integration: Deep Q-Network Application

This slide shows the full system loop.

The software agent plays CartPole. There are two tasks: task 0 is the standard environment, and task 1 is the modified environment.

When the active task changes, the task switch selects the corresponding physical equation. In hardware terms, this is like choosing the magnetic configuration that exposes w0 or w1.

The neural network then performs backpropagation. It computes the Bellman target and the loss, but instead of using a normal optimizer, it extracts the gradient signs.

Those gradient signs update the internal hardware state x of specific crosspoints.

So the system has both software and hardware logic. The software computes the learning signal. The hardware-inspired controller decides how that signal changes the conductance states.

Slide 14 - The Synaptic Controller: Translating Physics into Code

The synaptic controller is the bridge between the software DQN and the physical synapse model.

On the software side, I use PyTorch and Gymnasium. The agent still looks like a normal DQN agent.

But behind every trainable scalar parameter, there is a MultiWeightSynapse. This means each weight and bias in the network is backed by an internal physical state.

The controller has two main operations.

The first is load_weights(ap_index). This function selects the active task and converts the internal conductance states into effective PyTorch weights.

The second is step(ap_index). This applies the gradient-sign update to the internal synaptic states.

The ap_index works like a physical toggle switch. If it is 0, the network loads the task 0 effective weights. If it is 1, it loads the task 1 effective weights.

Slide 15 - The Validation Environment: Dual CartPole Realities

To validate the method, I used two CartPole environments.

Task 0 is the standard CartPole. The pole length is 0.5, with standard physical parameters.

Task 1 is a modified CartPole. Here the pole length is 0.7, and the mass is also distinct.

Both tasks have the same state representation. The state includes cart position, cart velocity, pole angle, and pole angular velocity. Both tasks also have the same action space: move left or move right.

This is important because the same neural-network architecture can be used for both tasks.

But the physics is different. The same action can produce slightly different behavior in the two environments.

So the challenge is: can one shared network architecture adapt to both tasks by changing only the physical task index in the synapse?

Slide 16 - Algorithmic Architecture: DQN with Task-Specific Memory

This slide shows the algorithmic structure.

The input is the CartPole state vector with four values. The network has two fully connected hidden layers with LeakyReLU activation. The output gives two Q-values: one for moving left and one for moving right.

The action with the higher Q-value is selected during exploitation.

Each task has its own replay buffer. This is important because the two environments have different dynamics.

The agent also uses epsilon-greedy exploration. At the beginning, epsilon is high, so the agent tries many random actions. Over time, epsilon decreases, and the agent relies more on the learned Q-values.

So the architecture is standard DQN in structure, but the weight storage and update rule are hardware-aware.

Slide 17 - Training Performance: Overcoming Hardware-Induced Instability

Here we see the training curves.

At the beginning, the reward is unstable. This is expected. The agent is still exploring because epsilon is high.

Over time, the reward improves. Eventually, both tasks reach the maximum reward of 100.

The important result is not that the curve is perfectly smooth. The important result is that the hardware-aware update still reaches a successful policy for both tasks.

Slide 18 - Policy Validation: Proof of Stable Generalization

For testing, exploration is turned off. That means epsilon is zero.

The agent no longer chooses random actions. It only chooses the action with the highest predicted Q-value.

This is important because it tests the learned policy itself, not random exploration.

The results are very strong. Across 1,000 test episodes, both configurations consistently achieve the maximum reward of 100.

This means the learned behavior is stable. The discrete spintronic synapse model did not prevent the network from learning a robust control policy.

So this slide confirms that the method can solve both environments after training.

Slide 19 - Visualizing the Brain: Divergent Weight Patterns

This slide visualizes the learned weight matrices.

On the left, we see the final layer weights for the standard task, with pole length 0.5. On the right, we see the same layer for the modified task, with pole length 0.7.

The architecture is identical. The number of neurons is the same. The input and output sizes are the same.

But the learned weight patterns are not the same.

The modified task shows stronger and more structured activations. This makes sense because the dynamics are different. A longer or heavier pole changes how the system responds to actions, so the network needs different internal features.

Slide 20 - The Path Forward for In-Memory AI

To conclude, this work shows that hardware-aware reinforcement learning using conductance-based synaptic models is feasible for continuous control tasks.

The results prove that a DQN agent can learn two related tasks using a multi-weight synaptic model and a sign-based hardware-compatible update rule.

At the same time, there are still engineering challenges. Real devices have nonlinear updates. They can have programming noise. Conductance can drift over time. Training may also become sensitive to initialization and network architecture.

So the message is not that the hardware is already perfect. The message is that the direction is promising.

This architecture points toward low-power, scalable, in-memory AI systems, especially for edge computing, where energy efficiency is very important.

The final takeaway is this: by respecting the physics of the device instead of ignoring it, we can design learning algorithms that are more compatible with future AI hardware.